

CO5_ASSESSMENTS 2

PSEUDOCODE WRITING TASK

1. Real-Time Face Detection and Recognition

A security system must process a live camera stream to:

- Capture video frames continuously.
- Detect multiple faces.
- Recognize registered persons using a trained dataset.
- Display bounding boxes and identity labels.

Constraints:

- Must operate in real time.
- Processing delay should be minimized.
- Multiple faces must be handled simultaneously.

Write detailed pseudocode for the complete OpenCV-based face detection and recognition pipeline. Include image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control.

2. Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to:

- Detect vehicles entering a predefined region of interest.
- Track multiple vehicles across consecutive frames.
- Count vehicles crossing a virtual line.
- Display bounding boxes, tracking IDs, and vehicle count.

Constraints:

- Avoid double counting.
- Handle multiple vehicles simultaneously.
- Maintain near real-time performance.

Write detailed pseudocode for the complete OpenCV-based vehicle detection, tracking, and counting system. Include video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

ANSWERS

1. Real-Time Face Detection and Recognition

Aim:

To design an OpenCV-based system that captures live video, detects multiple faces, recognizes registered persons, and displays their identities in real time.

Detailed Pseudocode

BEGIN

1. INITIALIZATION

- Load OpenCV library
- Load trained face detection model
- Load trained face recognition model
- Load registered person's face feature dataset
- Set recognition threshold
- Set camera resolution and frame rate

2. OPEN CAMERA

- Start video capture from camera
- IF camera is not available THEN
 - Display "Camera Error"
 - STOP
- END IF

3. CONTINUOUS VIDEO LOOP

- WHILE camera is open
 - Capture one frame from camera
 - IF frame is not successfully captured THEN
 - BREAK
- END IF

4. PREPROCESSING

- Resize frame to suitable resolution
- Convert frame to grayscale if required
- Apply noise reduction/filtering
- Improve image contrast if necessary

5. FACE DETECTION

- Detect all faces in the current frame
- FOR each detected face
 - Get bounding box coordinates
 - Crop the face region

6. FACE NORMALIZATION

- Resize cropped face to required size
- Align face using facial landmarks if available
- Normalize the face image

7. FEATURE EXTRACTION

- Extract facial features/face embedding
- Compare extracted features with registered feature dataset

8. FACE RECOGNITION

- Calculate similarity/distance between detected face and registered faces
- IF similarity is above threshold THEN
 - Assign registered person's name
- ELSE
 - Assign label "Unknown"
- END IF

9. VISUALIZATION

- Draw bounding box around detected face
- Display person's name above bounding box
- Display recognition confidence if required

END FOR

10. DISPLAY RESULT

- Show processed frame on screen

11. KEYBOARD CONTROL

- Check whether exit key is pressed
- IF exit key is pressed THEN
- BREAK
- END IF

12. PERFORMANCE OPTIMIZATION

- Process resized frames
- Avoid unnecessary image conversions
- Use lightweight detection/recognition models
- Detect faces periodically and track between detections
- Release unused memory

END WHILE

13. TERMINATION

- Release camera
- Close all OpenCV windows
- Clear allocated resources

END

Flow:

Camera → Frame Capture → Preprocessing → Face Detection → Face Alignment
→ Feature Extraction → Recognition → Bounding Box & Label → Display →
Next Frame

Efficiency Techniques:

- Resize frames before processing.
- Use lightweight OpenCV-compatible models.
- Process only the required face regions.
- Use tracking between detection frames.
- Avoid storing unnecessary frames.
- Maintain a fixed processing resolution.
- Exit immediately when the user presses the exit key.

Conclusion:

The pseudocode provides a complete real-time face recognition pipeline. Multiple faces are processed in each frame, while resizing, lightweight models, and efficient loop control help minimize processing delay.

2. Vehicle Detection, Tracking and Counting

Aim:

To design an OpenCV-based system that detects vehicles, tracks multiple vehicles, counts vehicles crossing a virtual line, and displays their IDs in real time.

Detailed Pseudocode

BEGIN

1. INITIALIZATION

- Load OpenCV library
- Open traffic video/camera
- Define Region of Interest (ROI)
- Define virtual counting line
- Initialize vehicle detector
- Initialize multi-object tracker
- Set minimum detection confidence
- Set minimum vehicle area
- Set vehicle count = 0
- Create set of counted vehicle IDs

2. OPEN VIDEO

- IF video/camera cannot be opened THEN
 - Display "Video Error"
 - STOP
- END IF

3. CONTINUOUS VIDEO LOOP

- WHILE video is open
 - Capture current frame
 - IF frame is not available THEN
 - BREAK
- END IF

4. PREPROCESSING

- Resize frame
- Apply Gaussian/median filtering if required
- Select the predefined ROI

5. VEHICLE DETECTION

- Detect vehicles inside ROI
- FOR each detected vehicle
 - Get bounding box
 - Calculate detection confidence
 - IF confidence is below threshold THEN
 - Ignore detection
 - ELSE
 - Store vehicle bounding box
 - END IF
- END FOR

6. VEHICLE TRACKING

- Send detected vehicle positions to tracker
- Update tracker using current frame
- FOR each tracked vehicle
 - Assign/maintain unique tracking ID
 - Obtain current bounding box
 - Calculate center point
- END FOR

7. COUNTING LOGIC

- FOR each tracked vehicle
 - Get previous center position
 - Get current center position
 - IF vehicle crosses virtual counting line THEN
 - IF vehicle ID is NOT in counted vehicle IDs THEN
 - vehicle count = vehicle count + 1
 - Add vehicle ID to counted vehicle IDs
 - END IF

END IF
END FOR

8. VISUALIZATION

Draw ROI boundary
Draw virtual counting line
FOR each tracked vehicle
 Draw bounding box
 Display tracking ID
 Display vehicle center point
END FOR
Display total vehicle count

9. PERFORMANCE OPTIMIZATION

Process resized frames
Process only ROI instead of entire frame
Use lightweight vehicle detector
Track vehicles between detection frames
Remove expired tracking IDs
Avoid unnecessary image processing

10. DISPLAY FRAME

Show processed frame

11. LOOP TERMINATION

Check keyboard input
IF exit key is pressed THEN
 BREAK
END IF
END WHILE

12. TERMINATION

Release video/camera

Destroy all OpenCV windows

Release tracker resources

END

Vehicle Counting Logic:

The important part is to **store counted vehicle IDs**.

IF vehicle crosses counting line THEN

IF vehicle_ID NOT IN counted_IDs THEN

count = count + 1

Add vehicle_ID to counted_IDs

END IF

END IF

This prevents the same vehicle from being counted multiple times.

Flow:

Camera → Frame Capture → Preprocessing → ROI → Vehicle Detection →
Multi-Vehicle Tracking → Line Crossing → ID Verification → Count Update →
Visualization → Next Frame

Handling Multiple Vehicles:

Each vehicle receives a unique tracking ID.

Example:

Vehicle 1 → ID 1

Vehicle 2 → ID 2

Vehicle 3 → ID 3

Vehicle 4 → ID 4

The tracker maintains these IDs across consecutive frames. Therefore, vehicles can be counted independently even when several vehicles appear simultaneously.

Efficient Loop Control:

- Process only the ROI.
- Resize frames.
- Use detection at selected intervals.
- Track vehicles between detections.
- Use a lightweight tracker.
- Terminate immediately when the exit key is pressed.
- Release camera and OpenCV resources after termination.

Conclusion:

The proposed pseudocode provides a complete vehicle detection, tracking, and counting pipeline. **Unique tracking IDs and a counted-ID set prevent double counting**, while ROI processing, frame resizing, and tracking-based optimization help maintain near real-time performance.